

UnifAI / Raksha Enterprise Server Implementation Guide

Comprehensive Production Deployment & Operations Manual

Document Version: 2.4.0

Target Systems: Ubuntu 22.04/24.04 LTS, RHEL 9, Debian 12, Docker Engine 24+

Classification: Enterprise Infrastructure & Systems Administration

Target Audience: DevOps Engineers, Site Reliability Engineers (SRE), IT Infrastructure Leads

1. System Requirements & Capacity Planning

1.1 Hardware Specifications

Workload Tier	Concurrent Users	Daily Requests	Recommended CPU	Recommended RAM	Storage (SSD NVMe)
Starter / PoC	1 – 50	< 10,000	2 vCPU	4 GB	25 GB
Mid-Market	50 – 500	10k – 200,000	4 vCPU	8 – 16 GB	100 GB
Enterprise Fleet	500 – 5,000+	200k – 2M+	8 – 16 vCPU	32 – 64 GB	500 GB+

1.2 Operating System & Kernel Requirements

- **Supported Linux Distributions:** Ubuntu 22.04 LTS / 24.04 LTS (Recommended), RHEL 9.x, Rocky Linux 9, Debian 12.
- **Kernel Tuning:**
 - File descriptor limit (**fs.file-max**): `>= 65536`
 - Conntrack table size (**net.netfilter.nf_conntrack_max**): `>= 131072`
 - Ephemeral port range (**net.ipv4.ip_local_port_range**): `1024 65535`
 - TCP keepalive settings: **tcp_keepalive_time = 300, tcp_keepalive_intvl = 15, tcp_keepalive_probes = 5**

1.3 Network & Firewall Port Allocation

Port	Protocol	Default Bind	Exposure	Purpose
80	TCP	0.0.0.0	Public Internet	HTTP redirect to HTTPS (Certbot / Nginx)
443	TCP	0.0.0.0	Public Internet	HTTPS public endpoint (Reverse Proxy)
6000	TCP	127.0.0.1	Localhost / Internal	UnifAI Go Backend Core Service (APP_PORT)

Port	Protocol	Default Bind	Exposure	Purpose
18182	TCP	0.0.0.0	LAN / VPN Only	Docker Network Proxy (Optional profile network-proxy)
5432	TCP	127.0.0.1	Localhost / Private	PostgreSQL Database Service
11434	TCP	127.0.0.1	Localhost / Internal	Ollama Local LLM Server (Optional AI Guard Bot)
18103	TCP	127.0.0.1	Client Laptop Only	Browser Guard Local MitM Proxy (Bound to loopback)
18195	TCP	127.0.0.1	Client Laptop Only	Browser Guard Local PAC Daemon (Bound to loopback)

2. Server Installation: Bare-Metal / Systemd Deployment

For high-security or bare-metal enterprise environments running without Docker containers, UnifAI can be deployed directly as a native Linux systemd daemon.

2.1 Directory Structure Setup

Execute as **root** or an administrative user with **sudo**:

```
# Create dedicated unifai system user and group sudo useradd -r -s /bin/false -d /opt/unifai unifai # Create application directories sudo mkdir -p /opt/unifai/{bin,configs,data,logs,release} sudo chown -R unifai:unifai /opt/unifai sudo chmod 750 /opt/unifai
```

2.2 Installing the Binary & Assets

```
# Copy compiled server executable to binary path sudo cp transports/raksha-http/unifai-server /opt/unifai/bin/unifai-server sudo chmod 755 /opt/unifai/bin/unifai-server # Copy default configs and branding assets sudo cp -r configs/* /opt/unifai/configs/ sudo cp -r apps/browser-guard/release/* /opt/unifai/release/ sudo chown -R unifai:unifai /opt/unifai
```

2.3 Systemd Service Definition

Create the unit file at **/etc/systemd/system/unifai.service**:

```
[Unit] Description=UnifAI Enterprise AI Gateway & Guard Service After=network.target postgresql.service Wants=postgresql.service [Service] Type=simple User=unifai Group=unifai WorkingDirectory=/opt/unifai EnvironmentFile=/opt/unifai/.env ExecStart=/opt/unifai/bin/unifai-server Restart=always RestartSec=5s LimitNOFILE=65536 LimitNPROC=4096 # Sandboxing and security hardening ProtectSystem=full ProtectHome=true NoNewPrivileges=true PrivateTmp=true # Standard log routing to journald StandardOutput=journal StandardError=journal [Install] WantedBy=multi-user.target
```

Reload systemd and enable the service:

```
sudo systemctl daemon-reload sudo systemctl enable unifai.service sudo systemctl start unifai.service sudo systemctl status unifai.service
```

3. Environment Configuration Breakdown (.env)

Every operational parameter must be declared in `/opt/unifai/.env` (or project root `.env`). Live secrets must never be committed to git.

```
# ===== # SECTION 1: PUBLIC DOMAIN
& NETWORK ADDRESSING # ===== # The
canonical public HTTPS address of your instance. Used for CORS, OAuth callbacks, # and generating Browser
Guard PAC discovery URLs. SERVER_DOMAIN=https://unifai.yourcompany.com # Listening port and interface for
the Go backend APP_PORT=6000 APP_HOST=0.0.0.0 # Storage directory for runtime configs, logs, and attachments
APP_DIR=/app/data # Container and Network Proxy names CONTAINER_NAME=raksha_tech PROXY_PORT=18182
NETWORK_PROXY_CONTAINER_NAME=raksha_browser_ai_proxy DOCKER_NETWORK=raksha-network #
===== # SECTION 2: DATABASE
PERSISTENCE (POSTGRESQL) # =====
DB_TYPE=postgres DB_HOST=127.0.0.1 DB_PORT=5432 DB_NAME=unifai_db DB_USER=unifai_user
DB_PASSWORD=SuperStrongDBPassword_ReplaceInProduction! # Set to 'require' or 'verify-full' in production
when Postgres has TLS enabled DB_SSL_MODE=disable #
===== # SECTION 3: INITIAL
BOOTSTRAP ADMIN CREDENTIALS # =====
# Used ONLY to seed the initial root administrator on first boot. ADMIN_EMAIL=security-admin@yourcompany.com
ADMIN_PASSWORD=ChangeMeImmediatelyAfterFirstLogin_987#@! #
===== # SECTION 4: CRYPTOGRAPHY,
SECRETS & INTEGRITY (CRITICAL) #
===== # Master encryption key for
database fields at rest (AES-256-GCM). # Generate with: openssl rand -base64 32
UNIFAI_ENCRYPTION_KEY=w3dF9kJ+z1bT5L0k8Q9w2e4r6t8y0u2i4o6p8a0s2d4= # Fleet-wide secret shared between
Browser Guard agents and server. # Generate with: openssl rand -base64 32
RAKSHA_GUARD_SECRET=G8kL0mN2p4Q6r8S0t2V4w6X8y0A2b4C6d8E0f2G4h6I= # When 1: Enforces fail-closed rejection if
guard secret is missing or invalid. RAKSHA_GUARD_REQUIRE_SECRET=1 # Anti PAC-hijacking flag. Prevents clients
from injecting arbitrary proxy targets. RAKSHA_PAC_ALLOW_QUERY_PROXY=0 # Secret for signing password-reset
tokens (HMAC-SHA256, minimum 32 chars). # Generate with: openssl rand -base64 48
PASSWORD_RESET_SECRET=J9kL2mN4p6Q8r0S2t4V6w8X0y2A4b6C8d0E2f4G6h8I0j2K4l6M8n0P2q4R6s8T0= # Multi-node cluster
replication secret (when deploying behind a load balancer)
CLUSTER_REPLICATE_SECRET=K1mN3p5Q7r9S1t3V5w7X9y1A3b5C7d9E1f3G5h7I9j1K3l5M7n9P1q3R5s7T9u1= #
===== # SECTION 5: REVERSE PROXY &
SECURITY HEADERS # ===== # Enable
X-Forwarded-Proto and X-Forwarded-For evaluation from trusted reverse proxy TRUST_PROXY_HEADERS=1 #
Comma-separated list of trusted upstream proxy CIDRs (e.g. 127.0.0.1, 10.0.0.0/8)
TRUSTED_PROXIES=127.0.0.1,172.16.0.0/12 #
===== # SECTION 6: OPTIONAL AI
GUARD BOT (OLLAMA) & BRANDING #
=====
OLLAMA_URL=http://127.0.0.1:11434 RAKSHA_COMPANY_NAME=Your Company Name
RAKSHA_COMPANY_LOGO=/your-company-logo.png # Logging format and level LOG_LEVEL=info LOG_STYLE=json
```

4. Reverse Proxy Setup (Nginx & Caddy Production Configurations)

The Go backend must run behind a production-hardened reverse proxy terminating TLS, managing HTTP/2, enforcing security headers, and handling WebSocket upgrades.

4.1 Production Nginx Configuration

Create `/etc/nginx/sites-available/unifai.conf`:

```
# Upstream definition for UnifAI Go backend upstream unifai_backend { server 127.0.0.1:6000; keepalive 64; }
# Redirect all plain HTTP traffic to HTTPS server { listen 80; listen [::]:80; server_name
unifai.yourcompany.com; # Let's Encrypt challenge path location /.well-known/acme-challenge/ { root
/var/www/certbot; } location / { return 301 https://$host$request_uri; } } # Primary HTTPS Production Server
server { listen 443 ssl http2; listen [::]:443 ssl http2; server_name unifai.yourcompany.com; # TLS
Certificate paths (Let's Encrypt or Custom CA) ssl_certificate
/etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem; # Modern TLS cipher suite ssl_protocols TLSv1.2
TLSv1.3; ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384
:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384; ssl_prefer_server_ciphers
off; ssl_session_timeout 1d; ssl_session_cache shared:SSL:10m; ssl_session_tickets off; # Security Headers
add_header X-Content-Type-Options nosniff always; add_header X-Frame-Options SAMEORIGIN always; add_header
X-XSS-Protection "1; mode=block" always; add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload" always; add_header Referrer-Policy "strict-origin-when-cross-origin" always; #
Maximum payload size for file uploads (documents, images, guard packages) client_max_body_size 100M;
client_body_buffer_size 128k; # Proxy buffer settings for fast streaming responses (SSE) proxy_buffering off;
proxy_cache off; proxy_read_timeout 600s; proxy_send_timeout 600s; proxy_connect_timeout 60s; # Root proxy
location location / { proxy_pass http://unifai_backend; proxy_http_version 1.1; # WebSocket upgrade headers
proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection $connection_upgrade; # Standard client
forwarding headers proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header
X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto https; proxy_set_header
X-Forwarded-Host $host; proxy_set_header X-Forwarded-Port 443; } # Browser Guard download and installer
assets caching location /api/browser-ai/download/ { proxy_pass http://unifai_backend; proxy_http_version
1.1; proxy_set_header Host $host; proxy_set_header X-Forwarded-Proto https; add_header Cache-Control
"private, no-cache, no-store, must-revalidate"; } } # Required for WebSocket connection upgrades in Nginx
http context # Add this inside /etc/nginx/nginx.conf if not already present: # map $http_upgrade
$connection_upgrade { # default upgrade; # '' close; # }
```

Enable configuration and test Nginx:

```
sudo ln -s /etc/nginx/sites-available/unifai.conf /etc/nginx/sites-enabled/ sudo nginx -t sudo systemctl
reload nginx
```

4.2 Alternative: Caddyfile Configuration (Zero-Config HTTPS)

If using Caddy as the reverse proxy:

```
unifai.yourcompany.com { reverse_proxy 127.0.0.1:6000 { header_up Host {host} header_up X-Real-IP
{remote_host} header_up X-Forwarded-Proto https transport http { read_buffer 0 write_buffer 0 } } }
```

5. Operations, Health Monitoring & Upgrades

5.1 Service Health Verification

Test the service endpoints using **curl**:

```
# Gateway Health Check curl -v http://127.0.0.1:6000/health # Expected HTTP 200 OK:
{"status":"ok","time":"..."} # Prometheus Metrics Check (if token enabled) curl -v -H "Authorization: Bearer
<RAKSHA_METRICS_TOKEN>" http://127.0.0.1:6000/metrics
```

5.2 Log Management with **journalctl** & Logrotate

When using systemd, view real-time logs with:

```
# Follow real-time server output sudo journalctl -u unifai.service -f # Filter for errors or fatal events
sudo journalctl -u unifai.service -p err..emerg -n 100
```

5.3 Zero-Downtime Rolling Upgrades

To upgrade the binary without dropping active client connections:

```
# 1. Download or compile the new binary sudo cp unifai-server-v2.5.0 /opt/unifai/bin/unifai-server.new sudo  
chmod 755 /opt/unifai/bin/unifai-server.new # 2. Swap binary atomically sudo mv  
/opt/unifai/bin/unifai-server.new /opt/unifai/bin/unifai-server # 3. Restart service with minimal latency  
sudo systemctl restart unifai.service # 4. Verify startup sudo journalctl -u unifai.service -n 50 --no-pager
```

End of Server Implementation Guide.